FastAPI Movie Backend
Complete Project Setup Guide

A step-by-step, industry-style guide to building a production-ready
Movie Management REST API with FastAPI, SQLAlchemy 2.x, Alembic, PostgreSQL & uv

Source: github.com/P123yu/fast_api_notes
Stack: FastAPI · SQLAlchemy 2.x · Alembic · Pydantic Settings · PostgreSQL · uv
Architecture: Router → Service → Repository → Model → Database

Table of Contents

1. Project Initialization

We start by creating the project directory and initializing it with **uv** — a modern, extremely fast Python package and project manager.

Create directories and open in VS Code

```
mkdir python_fast_api
cd python_fast_api
mkdir movie_backend
cd movie_backend
code .
```

Step 1 — Initialize with uv

From the project root (`movie_backend`):

```
uv init
```

Install core dependencies

```
uv add "fastapi[standard]" sqlalchemy alembic \
       "python-jose[cryptography]" argon2-cffi \
       pydantic-settings "psycopg[binary]"
```

Packages explained:
• **fastapi[standard]** — FastAPI + Uvicorn + common extras
• **sqlalchemy** — ORM (2.x style with Mapped / mapped_column)
• **alembic** — Database migration tool
• **python-jose[cryptography]** — JWT handling (for future auth)
• **argon2-cffi** — Modern password hashing
• **pydantic-settings** — Type-safe configuration from .env
• **psycopg[binary]** — PostgreSQL driver

2. Create main.py & Health Endpoint

Create the application package structure and a minimal FastAPI entry point.

Target structure

```
src/
■■■ movie_backend/
    ■■■ main.py
```

Content of main.py

```
from fastapi import FastAPI

app = FastAPI(
    title="Movie Backend API",
    description="Industry-style Movie Management REST API",
    version="1.0.0",
)


@app.get("/health")
def health_check():
    return {
        "status": "UP",
        "service": "movie-backend",
    }
```

This gives us a solid foundation: a FastAPI application with metadata and a simple health-check endpoint that can be used by load balancers or monitoring systems.

3. Run the Application

Because the package lives under `src/movie_backend`, the module path is `movie_backend.main:app`.

```
uv run uvicorn movie_backend.main:app --reload
```

Verify

Open **http://127.0.0.1:8000/docs**. You should see the interactive Swagger UI with the `GET /health` endpoint. Calling it returns:

```
{
    "status": "UP",
    "service": "movie-backend"
}
```

4. Core Configuration (Pydantic Settings)

Configuration and secrets must stay outside source code. We use **pydantic-settings** to load values from a `.env` file in a type-safe way.

Create core package

```
src/movie_backend/
■■■ core/
■   ■■■ __init__.py
■   ■■■ config.py
■■■ __init__.py
■■■ main.py
```

config.py

```
from pydantic_settings import BaseSettings, SettingsConfigDict

class Settings(BaseSettings):
    APP_NAME: str = "Movie Backend API"
    APP_VERSION: str = "1.0.0"
    DEBUG: bool = True
    DATABASE_URL: str

    model_config = SettingsConfigDict(
        env_file=".env",
        extra="ignore"
    )

settings = Settings()
```

.env (project root — NOT inside src)

```
APP_NAME=Movie Backend API
APP_VERSION=1.0.0
DEBUG=True
DATABASE_URL=postgresql+psycopg://postgres:password@localhost:5432/movie_db
```

Configuration flow

.env

↓

Pydantic Settings
↓
settings object
↓
Application code

Anywhere in the application you can now do:
`from movie_backend.core.config import settings`
`print(settings.DATABASE_URL)`

5. Database Layer (SQLAlchemy)

Create a dedicated `db` package that owns the engine, session factory, declarative base, and a FastAPI dependency for request-scoped sessions.

Structure

```
src/movie_backend/
■■■ core/
■   ■■■ __init__.py
■   ■■■ config.py
■■■ db/
■   ■■■ __init__.py
■   ■■■ database.py
■■■ __init__.py
■■■ main.py
```

database.py — full content

```
from sqlalchemy import create_engine
from sqlalchemy.orm import DeclarativeBase, sessionmaker

from movie_backend.core.config import settings

engine = create_engine(
    settings.DATABASE_URL,
    echo=settings.DEBUG,
)

SessionLocal = sessionmaker(
    bind=engine,
    autoflush=False,
    autocommit=False,
)


class Base(DeclarativeBase):
    pass


def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()


def test_connection():
    try:
        with engine.connect():
            print("Database connection successful!")
    except Exception as e:
```

```
        print(f"Database connection failed: {e}")
```

What each piece does

engine — SQLAlchemy's connection interface to PostgreSQL.
SessionLocal — factory that creates a Session (unit of work) per request.
Base — declarative base; all models inherit from it so they register in `Base.metadata`.
get_db() — FastAPI dependency that yields a session and guarantees it is closed.

Request lifecycle

Request arrives
↓
get_db() creates Session
↓
Route uses Session
↓
finally block runs
↓
Session is closed

Test the connection

```
uv run python -c "from movie_backend.db.database import test_connection; test_connection()"
```

Expected output: **Database connection successful!**

6. Alembic Migrations Setup

Alembic is the standard tool for versioned, controlled schema changes. We never rely on `Base.metadata.create_all()` for production schema evolution.

Desired workflow

SQLAlchemy Model change
↓
alembic revision --autogenerate
↓
Migration file generated
↓
alembic upgrade head
↓
PostgreSQL updated

Initialize Alembic

Stop Uvicorn if running (Ctrl+C), then:

```
uv run alembic init migrations
```

Resulting layout

```
movie_backend/
■■■ migrations/
■   ■■■ versions/
■   ■■■ env.py
```

```
■   ■■■ README
■   ■■■ script.py.mako
■■■ src/
■   ■■■ movie_backend/
■       ■■■ core/
■       ■■■ db/
■       ■■■ __init__.py
■       ■■■ main.py
■■■ .env
■■■ alembic.ini
■■■ pyproject.toml
■■■ uv.lock
```

Configure migrations/env.py

1. Add imports at the top:
`from movie_backend.db.database import Base`
`from movie_backend.core.config import settings`

2. Replace `target_metadata = None` with:
`target_metadata = Base.metadata`

3. Immediately after `config = context.config` add:

```
config.set_main_option(
    "sqlalchemy.url",
    settings.DATABASE_URL,
)
```

Now Alembic reads the database URL from the same `.env` → Settings path that the application uses. No duplicated configuration.

7. Movie Model + First Migration

We introduce a clean layered structure and create the first real entity.

Target architecture

```
src/movie_backend/
■■■ core/
■   ■■■ __init__.py
■   ■■■ config.py
■■■ db/
■   ■■■ __init__.py
■   ■■■ database.py
■■■ model/
■   ■■■ __init__.py
■   ■■■ movie.py
■■■ repository/
■   ■■■ __init__.py
■   ■■■ movie_repository.py
■■■ router/
■   ■■■ __init__.py
■   ■■■ movie_router.py
■■■ schema/
■   ■■■ __init__.py
■   ■■■ movie.py
■■■ service/
■   ■■■ __init__.py
■   ■■■ movie_service.py
■■■ main.py
■■■ __init__.py
```

model/movie.py (SQLAlchemy 2.x style)

```
from datetime import datetime

from sqlalchemy import DateTime, Integer, String, Text, func
from sqlalchemy.orm import Mapped, mapped_column

from movie_backend.db.database import Base


class Movie(Base):
    __tablename__ = "movies"

    id: Mapped[int] = mapped_column(
        Integer, primary_key=True, autoincrement=True,
    )
    title: Mapped[str] = mapped_column(String(255), nullable=False)
    description: Mapped[str | None] = mapped_column(Text, nullable=True)
    genre: Mapped[str] = mapped_column(String(100), nullable=False)
    release_year: Mapped[int] = mapped_column(Integer, nullable=False)
    created_at: Mapped[datetime] = mapped_column(
        DateTime(timezone=True),
        server_default=func.now(),
        nullable=False,
    )
    updated_at: Mapped[datetime] = mapped_column(
        DateTime(timezone=True),
        server_default=func.now(),
        onupdate=func.now(),
        nullable=False,
    )
```

Important: In `migrations/env.py` also import the model so Alembic sees it in `Base.metadata`:
`from movie_backend.model.movie import Movie`
(or re-export from `model/__init__.py` and import the package).

Generate & apply the migration

```
# Check current state
uv run alembic check

# Autogenerate migration
uv run alembic revision --autogenerate -m "create movies table"

# Apply it
uv run alembic upgrade head

# Verify again
uv run alembic check
```

After a successful upgrade the PostgreSQL database contains a `movies` table with columns: id, title, description, genre, release_year, created_at, updated_at.

8. Clean Architecture Layers

The recommended request flow keeps concerns separated:

FastAPI Router (HTTP)
↓
Service (business logic)
↓
Repository (data access)
↓
SQLAlchemy Model

↓

PostgreSQL

- **Router** — HTTP concerns only (status codes, dependencies, request/response models)
- **Service** — business rules, orchestration
- **Repository** — pure database operations
- **Model** — SQLAlchemy entity (table mapping)
- **Schema** — Pydantic models for API contracts (never expose ORM models directly)

9. Pydantic Schemas

API request/response shapes live in the schema layer, completely separate from the SQLAlchemy model.

schema/movie.py

```
from pydantic import BaseModel


class MovieCreate(BaseModel):          # or MovieRequest
    title: str
    description: str | None = None
    genre: str
    release_year: int


class MovieResponse(BaseModel):
    id: int
    title: str
    description: str | None
    genre: str
    release_year: int

    class Config:
        from_attributes = True         # enables ORM mode (Pydantic v2)
```

`from_attributes = True` (Pydantic v2) allows creating a response model directly from a SQLAlchemy instance.

10. Repository, Service & Router

repository/movie_repository.py

```
from sqlalchemy.orm import Session
from movie_backend.model.movie import Movie


class MovieRepository:

    def create(self, db: Session, movie: Movie) -> Movie:
        db.add(movie)
        db.commit()
        db.refresh(movie)
        return movie

    def get_all(self, db: Session):
        return db.query(Movie).all()
```

service/movie_service.py

```
from sqlalchemy.orm import Session

from movie_backend.model.movie import Movie
```

```
from movie_backend.repository.movie_repository import MovieRepository


class MovieService:

    def __init__(self):
        self.repository = MovieRepository()

    def create_movie(self, db: Session, movie: Movie) -> Movie:
        return self.repository.create(db, movie)

    def get_all_movies(self, db: Session):
        return self.repository.get_all(db)
```

router/movie_router.py

```
from fastapi import APIRouter, Depends
from sqlalchemy.orm import Session

from movie_backend.db.database import get_db
from movie_backend.schema.movie import MovieCreate, MovieResponse
from movie_backend.model.movie import Movie
from movie_backend.service.movie_service import MovieService


router = APIRouter(
    prefix="/api/v1/movies",
    tags=["Movies"],
)

service = MovieService()


@router.post("/", response_model=MovieResponse)
def create_movie(
    movie: MovieCreate,
    db: Session = Depends(get_db),
):
    movie_model = Movie(**movie.model_dump())
    return service.create_movie(db, movie_model)


@router.get("/", response_model=list[MovieResponse])
def get_all_movies(
    db: Session = Depends(get_db),
):
    return service.get_all_movies(db)
```

11. Wire Everything in main.py

```
from fastapi import FastAPI

from movie_backend.router.movie_router import router as movie_router


app = FastAPI(
    title="Movie Backend API",
    description="Industry-style Movie Management REST API",
    version="1.0.0",
)

app.include_router(movie_router)


@app.get("/health")
def health_check():
```

```
    return {
        "status": "UP",
        "service": "movie-backend",
    }
```

Now you have fully working endpoints:
• **POST /api/v1/movies** — create a movie
• **GET /api/v1/movies** — list all movies
• **GET /health** — health check

12. Key Concepts & Points to Remember

SQLAlchemy 2.x style preference

Aspect	Column() style	Mapped + mapped_column()
SQLAlchemy 2.x	Supported	Supported
Type hints	Limited	Excellent
IDE support	Good	Better
Modern projects	Less preferred	**Preferred**
New project	Avoid	**Use this**

Why Alembic is required

Question: If I add a new field (e.g. `rating`) to an existing SQLAlchemy model and simply restart FastAPI, will the column appear in PostgreSQL?

> **Answer: No.** SQLAlchemy's `Base.metadata.create_all(engine)` only creates tables that do not yet exist. It does **not** alter existing tables when the model changes.

• Table does not exist → `create_all()` → table is created ■
• Table already exists + model gains a column → `create_all()` → nothing happens ■

In production projects Alembic is the standard choice because it gives controlled, versioned, reviewable migrations rather than silently modifying the database.

Full stack flow summary

HTTP Request
↓
FastAPI Router (validation via Pydantic schema)
↓
Service layer (business logic)
↓
Repository (SQLAlchemy operations)
↓
SQLAlchemy Model → Engine → psycopg
↓
PostgreSQL

This guide covers the complete foundation: project scaffolding with uv, configuration, SQLAlchemy 2.x, Alembic, layered architecture (Router → Service → Repository → Model), and a working CRUD-ready Movie API. From here you can extend with authentication, exception handling, testing, Docker, and production configuration.

Source notes: https://github.com/P123yu/fast_api_notes